



北京交通大学
BEIJING JIAOTONG UNIVERSITY



计算机科学与技术学院
School of Computer Science & Technology

第七章 指令系统

尹辉 刘万成

hyin@bjtu.edu.cn

wchliu@bjtu.edu.cn

办公地点：九教北312，401室

- 指令集架构的组成

- 指令的寻址方式

- CISC VS RISC

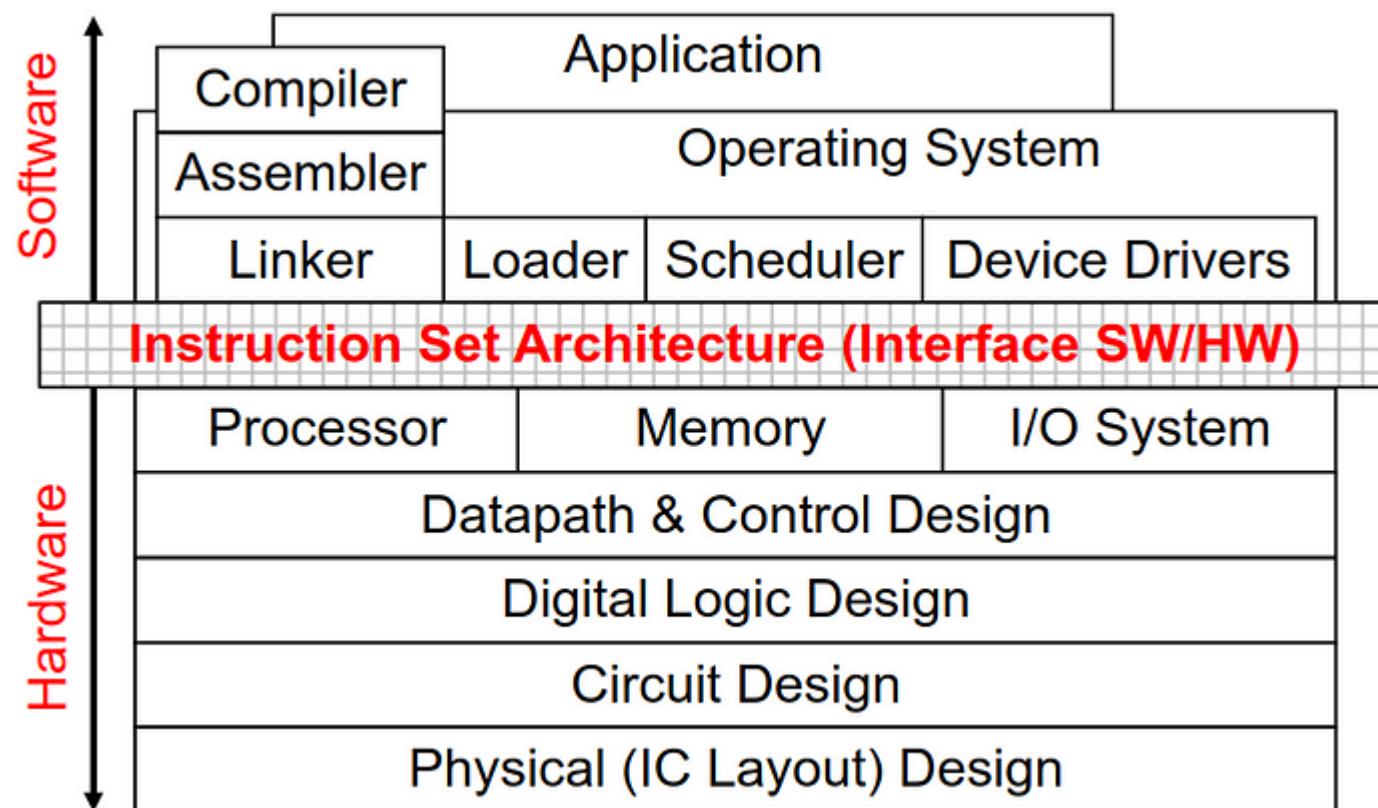
□ 指令集架构 (Instruction Set Architecture, ISA)

◆ 含义

(1) 计算机硬件与软件之间的核心接口规范

(2) 定义了CPU能够理解并执行的基本指令集合、寄存器结构、内存访问方式以及硬件支持的操作模式

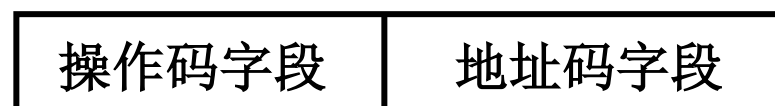
(3) 编译器或汇编程序员直接依赖的底层抽象，决定了软件如何控制硬件资源。



常见的指令集架构：x86/x86-64(Intel/AMD)、ARM、RISC-V、MIPS、LoongArch等

□ 机器指令

◆ 指令的一般格式



◆ 操作码：反映机器指令完成什么操作

- 定长操作码：便于硬件设计，译码时间短
- 变长操作码：操作码分散在指令字的不同字段中
增加了指令译码和分析的难度

◆ 地址码：反映机器指令操作数的来源

通常与寻址方式结合决定具体的操作数

指令集架构的组成

□ 机器指令：操作码扩展技术

4 位操作码

OP	A ₁	A ₂	A ₃
0000	A ₁	A ₂	A ₃
0001	A ₁	A ₂	A ₃
⋮	⋮	⋮	⋮
1110	A ₁	A ₂	A ₃

最多16条三地址指令

8 位操作码

1111	0000	A ₂	A ₃
1111	0001	A ₂	A ₃
⋮	⋮	⋮	⋮
1111	1110	A ₂	A ₃

三地址指令操作码
每减少一种可多构成 2^4 种二地址指令

12 位操作码

1111	1111	0000	A ₃
1111	1111	0001	A ₃
⋮	⋮	⋮	⋮
1111	1111	1110	A ₃

二地址指令操作码
每减少一种可多构成 2^4 种一地址指令

16 位操作码

1111	1111	1111	0000
1111	1111	1111	0001
⋮	⋮	⋮	⋮
1111	1111	1111	1111

16条零地址指令

要点1：每条机器指令中的操作码都是唯一的

要点2：可通过减少地址码来扩展操作码的位数

□ 机器指令：操作码扩展技术

举例：指令系统共有74种指令，前4种使用频率平均为0.12，中间15种使用频率平均为0.02，最后55种使用频率平均为0.004。如何编码？平均码长为多少？

平均码长的定义：
$$\sum_i F_i * B_i$$

i: 表示第i条指令

F_i : 指令的频率

B_i : 编码所用的位数

□ 机器指令：操作码扩展技术

举例：指令系统共有74种指令，前4种使用频率平均为0.12，中间15种使用频率平均为0.02，最后55种使用频率平均为0.004。如何编码？平均码长为多少？

解：

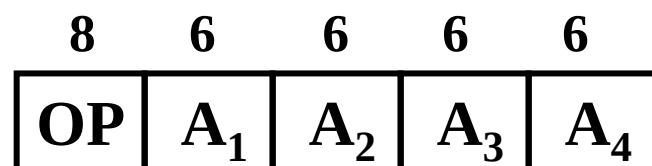
	指令数	频率
000 001 010 011	4	0.12
100-0000 100-1110	15	0.02
100-1111-000000 100-1111-110110	55	0.004

有没有更短的平均码长？

$$\text{平均码长} = 0.12 * 4 * 3 + 0.02 * 15 * 7 + 0.004 * 55 * 13 = 1.44 + 2.1 + 2.86 = 6.4$$

□ 机器指令：地址码
(1) 四地址

设指令字长为 32 位
操作码固定为 8 位



(A₁) OP (A₂) → A₃

A₁ 第一操作数地址 A₂ 第二操作数地址

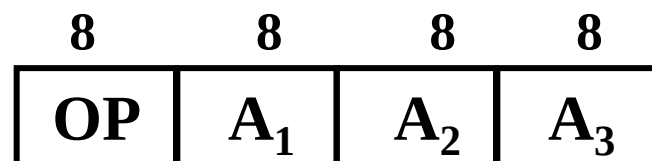
A₃ 结果的地址 A₄ 下一条指令地址

寻址范围 $2^6 = 64$

5 次访存

(取指令, 2次取2操作数,
存结果, 取指令地址)

(2) 三地址



(A₁) OP (A₂) → A₃

若 A₃ 用 A₁ 或 A₂ 代替

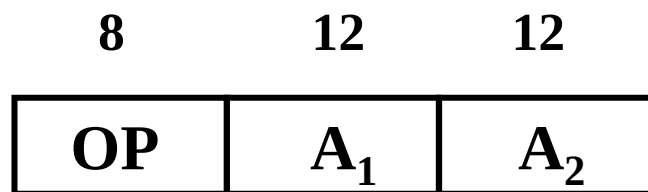
若 PC 代替 A₄

寻址范围 $2^8 = 256$

4 次访存(取指令, 2次
取2操作数, 存结果)

指令集架构的组成

□ 机器指令：地址码 设指令字长为 32 位
(3) 二地址 操作码固定为 8 位



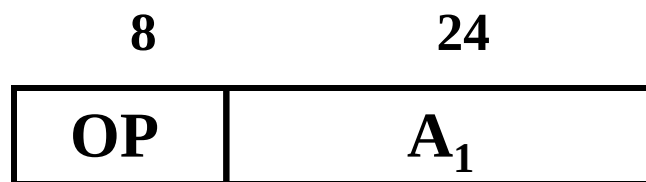
或
(A₁) OP (A₂) → A₁ ACC
(A₁) OP (A₂) → A₂ ACC

寻址范围 $2^{12} = 4 \text{ K}$

4 次访存(取指令，2次
取2操作数，存结果)

若结果存于寄存器ACC 3次访存

(4) 一地址



(ACC) OP (A₁) → ACC

若ACC 代替 A₁ (或A₂)

寻址范围 $2^{24} = 16 \text{ M}$

2 次访存

(5) 零地址 无地址码

□ 机器指令：指令字长

指令字长决定于 {
操作码的长度
操作数地址的长度
操作数地址的个数

(1) 指令字长 固定

指令字长 = 存储字长

(2) 指令字长 可变

按字节的倍数变化

特别注意：指令和数据在存储器中的存储

①大端存储：字地址 为 高字节 地址

	字地址		低字节	
0	0	1	2	3
4	4	5	6	7

②小端存储：字地址 为 低字节 地址

	字地址		低字节	
0	3	2	1	0
4	7	6	5	4

字地址 为 低字节 地址

指令集架构的组成

□ 机器指令：指令与数据的存储

设存储字长为 32位，小端存储

(1) 边界对准

地址（十进制）

字（地址 0）				0
字（地址 4）				4
字节（地址11）	字节（地址10）	字节（地址 9）	字节（地址 8）	8
字节（地址15）	字节（地址14）	字节（地址13）	字节（地址12）	12
半字（地址18）✓		半字（地址16）✓		16
半字（地址22）✓		半字（地址20）✓		20
双字（地址24）				24
双字				28
双字（地址32）				32
双字				36

存/取一个字需一次访存

(2) 边界未对准

地址（十进制）

字(地址2)		半字(地址0)	0
字节(地址7)	字节(地址6)	字(地址4)	4
半字(地址10)		半字(地址8)	8

存/取一个字需两次访存

□ 机器指令的类型

1. 数据传送

源	寄存器	寄存器	存储器	存储器
目的	寄存器	存储器	寄存器	存储器
如x86架构	MOVE	STORE MOVE PUSH	LOAD MOVE POP	MOVE

2. 算术逻辑操作

加、减、乘、除、增 1、减 1、浮点运算等

与、或、非、异或、位测试等

如 x86架构 **ADD SUB MUL DIV INC DEC**
 AND OR NOT XOR TEST

□ 机器指令的类型

3. 移位操作

算术移位 逻辑移位

循环移位（带进位和不带进位）

4. 转移

(1) 无条件转移 **JMP**

(2) 条件转移

结果为零转 ($Z = 1$) **JZ**

结果溢出转 ($O = 1$) **JO**

结果有进位转 ($C = 1$) **JC**

如 300
 $\vdots$
 305
 306
 307

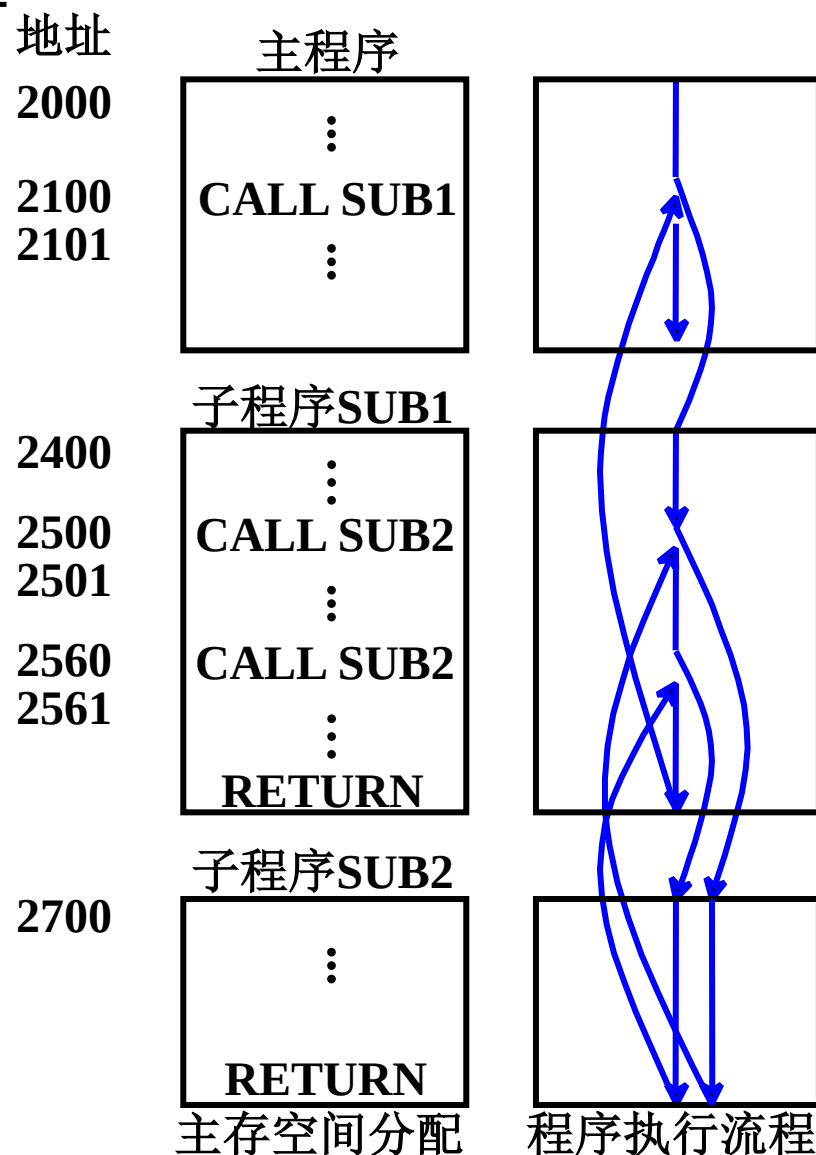
完成触发器

JZ D = 0 则跳

□ 机器指令的类型

5. 调用和返回

如x86



6. 输入输出

输入

端口地址 $\longrightarrow$ CPU 的寄存器

如x86 `IN AX, n`

输出

CPU 的寄存器 $\longrightarrow$ 端口地址

如x86 `OUT n, AX`

□ 机器指令的类型

7. 陷阱 (Trap) 指令

陷阱指令是一类用于主动触发处理器异常或模式切换的特殊指令，如系统调用、处理错误或调试事件

(1) 显式陷阱指令：由程序主动执行，直接触发异常。

示例： RISC-V: ECALL (环境调用)、EBREAK (断点调试)

x86: INT n (软件中断，如INT 0x80用于Linux系统调用)

ARM: SVC (Supervisor Call, 用于系统调用)

(2) 隐式陷阱条件：由指令执行过程中的错误或异常条件自动触发。

示例： 无效指令：执行未定义的指令编码

内存访问错误：访问未对齐地址或受保护内存区域

算术异常：除零错误或整数溢出

□指令集架构的操作模式 (Operating Modes)

- ◆ 用于管理资源访问权限、隔离代码执行环境，并增强系统安全性与稳定性。这些模式决定了处理器能执行哪些指令、访问哪些硬件资源（如内存、外设），是操作系统和应用程序之间权限控制的核心机制

(1) RISC-V架构

- **用户模式 (User Mode)** :运行应用程序，受限执行环境。
- **监督者模式 (Supervisor Mode)** :操作系统内核运行，处理系统调用和分页内存管理。
- **机器模式 (Machine Mode)** :最高权限，处理底层硬件操作（如启动、异常处理），通常由固件（如Bootloader）使用。

(2) ARM架构 (RISC) :

ARMv8/v9定义了异常级别 (Exception Levels, EL) , 从EL0到EL3四个层级，权限逐级升高：

- **EL0 (用户模式)** : 运行普通应用，权限最低。
- **EL1 (操作系统内核)** : 管理进程、内存等。
- **EL2 (虚拟机监控程序)** : 支持虚拟化（如Hypervisor）。
- **EL3 (安全监控模式)** : 处理安全世界与非安全世界的切换（如TrustZone技术）。

□指令集架构的寄存器

- ◆ 寄存器被内置在CPU内部
- ◆ 寄存器的优势：**速度极快**（工作速度小于1ns）
- ◆ 寄存器的数量是**设计均衡**的体现
 - 均衡的要素：性能与可用性
 - 数量少：结构简单，速度快，能够存储在CPU内的数据少
 - 数量多：结构复杂，速度慢，能够存储在CPU内的数据多

RISC-V指令集的通用寄存器

(1) 基本整数寄存器

RISC-V RV32I 定义 32个32位通用整数寄存器 (x0-x31)，其中：

- x0 寄存器：硬件固定为0（读恒为0，写无效），常用于简化指令设计（如清零或作为地址基址）。
- 其他寄存器 (x1-x31)：可自由读写，用于算术运算、逻辑操作和数据传输。

(2) 浮点寄存器（可选）

若支持浮点扩展（如F/D扩展），则提供 32个独立的32/64位浮点寄存器 (f0-f31)，用于单精度（F）或双精度（D）浮点运算。

- 指令集架构的组成

- 指令的寻址方式

- CISC VS RISC